Recycling Rewards

Center-verified hackathon MVP

Working name: LoopMint. Revised October 9, 2026.

Product and scope

Users bring recyclable materials to a participating center. Authorized center staff confirm physical receipt and accepted quantities. The Python backend uses that confirmation to calculate and distribute funded token rewards.

Verification policy: no Gemini integration, AI photo analysis, AI approval, or AI fallback. Center confirmation is the authoritative record of accepted recycling intake. A user submission, photo, or QR scan alone cannot authorize rewards.

- Mobile-friendly website; plastic bottles and dry cardboard; one participating center initially.
- Three roles: recycler, center verifier, and funding partner. Roles and center assignments are server-controlled.
- Wallet login, username, claim submission, unique short-lived QR ticket, and center review.
- Submission records selected center, material, and estimated quantity. A photo may be kept as optional supporting evidence; it is never required for automated analysis.
- Wallet shows pending and confirmed rewards, balance, transaction history, explorer links, and simulated redemption.
- Generate an optional post template from verified quantities and confirmed rewards, then prompt users to share it on their own social platforms. LoopMint does not host posts or a community feed.
- Impact dashboard shows bottle counts, cardboard weight, participation, and reward trends by center.
- Funding dashboard shows received funds, reserved budget, issued rewards, and pending reimbursements.

Government partnership, funding, and cash payouts remain simulated for the prototype. Use Solana Devnet and label tokens as having no real monetary value. Keep the original planning assumption of 36-48 hours with 3-4 teammates.

Center confirmation and token distribution

1. The recycler submits a claim and receives a short-lived QR ticket tied to that claim and the selected center.

2. Authenticated staff assigned to that center scan the ticket to open the claim. Scanning does not approve it.

3. Staff inspect the physical materials, apply center acceptance rules, and record actual accepted bottle count or cardboard weight. Unaccepted materials receive no reward.

4. Staff explicitly confirm receipt and approve or reject the claim. Save the claim, center, verifier, accepted quantities, timestamp, and decision. Staff cannot approve their own claims.

5. On approval, Python calculates the reward using the accepted quantities and the configured rate. Save the rate used with the claim.

6. In a database transaction, record the verification and a single reward entitlement per claim. Reserve available funding and create one issuance job. If funding is insufficient, keep the verified claim with its reward awaiting funding; issue no tokens yet.

7. A Python worker persists the signed Solana transaction and its signature before broadcasting. It checks network confirmation and reconciles uncertain results before retrying.

8. Show the reward as confirmed only after blockchain confirmation. Update wallet history, the funding ledger, and reward analytics from that result.

Claim states: submitted, awaiting review, rejected, verified. Reward states are separate: awaiting funding, queued, sent, confirmed. A rejected or unconfirmed claim cannot create an issuance job. A verified claim can still have an unpaid reward.

Protect the confirmation endpoint with server-side role and center checks. Use a claim lock or equivalent transactional state check, a unique reward per claim, and a unique issuance job per reward. Repeated or concurrent confirmations return the existing result without another reward. Funding release and retry processing must preserve those constraints.

Illustrative demo economics

- Accepted plastic bottles: 5 LOOP per verified bottle.
- Accepted dry cardboard: 20 LOOP per verified kilogram.
- Simulated redemption: 1 LOOP represents $0.01 of demo credit.
- Example: 10 bottles earn 50 LOOP; 2 kg of cardboard earn 40 LOOP; total 90 LOOP.

These are proposed demo settings, not government rates. Store weight as integer grams and money as integer cents. Define token precision and rounding before implementation, calculate in integer token base units, and save the applicable rate. Expected reimbursements do not count as available funding. Redemption requires a user-authorized, confirmed burn and an idempotent simulated payout record.

Post templates and external sharing

After center confirmation, show an optional "Share your impact" prompt. Generate a caption and a downloadable image card using the claim's accepted quantities. Include a reward amount only when the token transaction is confirmed; otherwise omit the reward line. Sharing never gates approval, token distribution, or redemption.

1. Show a preview of the post template with accepted materials and quantities. The user can edit the caption before sharing.

2. Offer "Share", "Copy caption", "Download card", and "Not now" actions.

3. Where supported, the share action opens the device's sharing interface so the user can choose a destination. Keep copying and downloading available for manual posting.

4. The user chooses their own social account and completes publication on the destination platform. LoopMint does not automatically publish or connect social accounts in the MVP.

Example post template

"I dropped off 2 kg of cardboard for recycling, confirmed by my recycling center. I received 40 LOOP demo tokens. Solana Devnet demo tokens - no real monetary value. #LoopMint #Recycling"

The image card shows "Center-confirmed recycling intake", "2 kg cardboard", and, once confirmed, "40 LOOP received". Include "Solana Devnet demo tokens - no real monetary value" whenever demo tokens appear in either the caption or card.

Data and privacy

Generate the caption and card on demand in the browser from the user's authorized claim and reward records. Keep edits local to the sharing preview; do not create a posts table, public post page, hosted feed, or server-side library of social cards. Private recycling evidence remains separate and is not included in the template. Omit wallet addresses, ticket QR codes, verifier identities, and private claim identifiers.

Closing the sharing interface does not establish that a post was published. Show accurate local feedback such as "Caption copied" or "Card downloaded" and do not track an external post as published without evidence. A canceled share leaves the recycling claim and reward unchanged.

Python architecture and records

Retain the original plan's proposed stack, with the AI service removed:

- Interface: Next.js, React, Tailwind CSS; Recharts with API polling for dashboards.
- Sharing interface: browser-generated caption and image card, preview, share action, copy, and download; no hosted social content or social account integration.
- API: Python and FastAPI for permissions, submissions, confirmations, and reward endpoints.
- Validation: Pydantic for request and response schemas, material units, and accepted quantities.
- Database: Tiger Cloud / TimescaleDB; Psycopg 3 for parameterized SQL and transactions.
- Token services: Solana.py and solders; browser wallet through Solana Kit and React bindings.
- Background work: Python worker and PostgreSQL jobs for issuance, retries, and reconciliation.
- Optional evidence: private Cloudflare R2 objects with short-lived signed access.
- Deployment and validation: separate frontend, API, and worker services on Railway; pytest and Playwright.

Service flow: recycler or center interface -> FastAPI -> PostgreSQL records and reward jobs -> Python worker -> Solana Devnet. Dashboards read authoritative API results. Keep issuer keys and credentials server-side.

Modules

- `routes/claims.py`: submissions and center confirmation endpoints, with access enforcement.
- `schemas.py`: claim, measured quantity, center decision, and reward response schemas.
- `services/verification.py`: center policies, physical receipt, verifier identity, and recorded decision.
- `services/rewards.py`: deterministic calculation, funding reservation, and unique reward/job creation.
- `services/solana_client.py`: build, send, and inspect token transactions.
- `repositories/claims.py`: parameterized queries and transactional claim state changes.
- `worker.py`: process funded reward jobs and reconcile interrupted transactions.

There is no Gemini client, AI schema, AI dependency, AI API key, or AI service in this architecture.

Records and analytics

- `users`, `centers`: wallet, username, roles, center assignment, and acceptance policies.
- `claims`, `claim_items`, `verifications`: ticket identity and expiry, optional evidence, submitted and accepted quantities, center, verifier, receipt timestamp, decision, and rejection reason.
- `reward_rules`: material, unit, rate, effective dates, and immutable rate snapshot on approval.
- `funding_ledger`: received funds, reservations, releases, and simulated payouts.
- `rewards`, `reward_jobs`, `redemptions`: unique claim/reward references, amount, state, signed transaction, signature, retry data, and redemption tracking.
- `recycling_events`: timestamped intake and reward events for time-series analytics.

Use ordinary PostgreSQL tables for authoritative state, funding, and unique constraints. Use a hypertable for analytics, with hourly aggregates by center and material. Count accepted intake once, independently of later reward events. Report bottles and cardboard weight separately. Label metrics as verified intake for recycling; center receipt does not establish downstream processing. Keep evidence private and sharing optional. Generate social cards from approved display fields, without private evidence or location metadata.

Build order and acceptance checks

1. Foundation: interface, FastAPI API, database connection, wallet login, server-controlled roles, and schema.

2. Center verification: claim submission, QR ticket, center review screen, measured quantities, recorded confirmation/rejection, and deterministic rewards.

3. Funded Solana rewards: atomic funding reservation, Devnet mint, Python worker, confirmation tracking, and wallet history.

4. Reliability: duplicate confirmation protection, center permissions, funding contention, expired tickets, and transaction recovery.

5. Sharing and metrics: post-template preview, external sharing prompt, copy/download options, event hypertable, impact charts, and funding dashboard.

6. Demo preparation: simulated redemption, failure-case checks, connected domain, and rehearsal.

Required acceptance checks

- Normal confirmation: accepted quantities produce the saved-rate reward, reserved funding, and one confirmed transaction.
- No center confirmation: a submission, uploaded image, ticket scan, or recycler request cannot authorize token distribution.
- Rejected claim: no reward issuance or funding reservation.
- Invalid confirmation: reject negative quantities, wrong units, unsupported materials, or a confirmation with no accepted quantity.
- Unauthorized verifier: reject recycler approval, another center's verifier, or self-approval without changing claim, funding, or reward records.
- Duplicate or concurrent confirmation: one reward entitlement, one funding reservation, and one issuance job per claim.
- Ticket reuse or expiry: no new approval from an already consumed or expired ticket; provide a controlled replacement process for an unverified claim.
- Insufficient funding: retain verified intake, mark the reward awaiting funding, and send no tokens until funding is reserved.
- Two claims use the last budget: transactional reservation prevents overspending.
- Worker crash or network timeout: inspect the persisted transaction signature before any new issuance; keep uncertain rewards pending.
- Repeated redemption: one simulated payout per verified burn transaction.
- Analytics: confirmation and reward events do not double-count materials.
- Sharing: use accepted quantities; show rewards only after transaction confirmation; include the demo-token label when applicable.
- External handoff: copy/download work when device sharing is unavailable; canceling or skipping sharing changes no claim or reward state and is not recorded as publication.

Demo and ownership

Demo: submit a 2 kg cardboard claim, show its QR ticket, switch to the assigned center verifier, record physical receipt and 2 kg accepted weight, and confirm. Show 40 LOOP after Devnet confirmation and open the explorer link. Preview the generated post template and demonstrate sharing to the user's chosen platform or copying the caption and downloading the card. Show impact and remaining budget. Repeat confirmation to demonstrate that no additional reward is issued.

Suggested ownership: backend handles permissions, confirmation, SQL, rewards, and tests; frontend handles recycler, center, wallet, and post-template sharing screens; integrations handles the Solana worker and recovery; analytics/demo handles events, dashboards, seed data, domain, and presentation. With three teammates, combine integrations and analytics/demo.

Retained demo tracks: sustainability through verified intake and participation; Solana through confirmed Devnet rewards; Tiger Data through event analytics; GoDaddy Registry through an eligible connected domain. Domain candidate: loopmint.club, with availability and event eligibility still unchecked. Remove Google Gemini from the sponsor track plan.

Keep native mobile apps, real government integrations, bank payouts, token trading, IoT bins, custom smart contracts, direct messaging, a general chatbot, hosted posts/feeds, and automatic social publishing outside the initial build. Add maps, challenges, and leaderboards later. If time is tight, preserve submission, center confirmation, one funded and confirmed reward, a copyable post template with a downloadable card, and a small impact dashboard.